# Solution: Biggest Island

## Problem Statement

Given a 2D array (i.e., a matrix) containing only 1s (land) and 0s (water), find the biggest island in it. Write a function to return the area of the biggest island.

An **island** is a connected set of 1s (land) and is surrounded by either an edge or 0s (water). Each cell is considered connected to other cells horizontally or vertically (not diagonally).

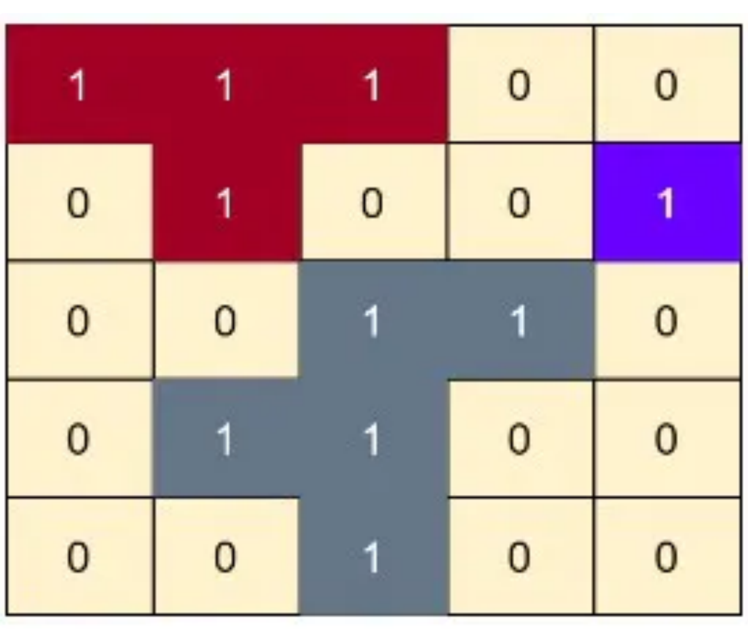
**Example 1**

**Input:** matrix =

| | | | | |
|---|---|---|---|---|
| 1 | 1 | 1 | 0 | 0 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 |
| 0 | 0 | 1 | 0 | 0 |

**Output:** 5

**Explanation:** The matrix has three islands. The biggest island has 5 cells .



| | | | | |
|---|---|---|---|---|
| 1 | 1 | 1 | 0 | 0 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 |
| 0 | 0 | 1 | 0 | 0 |

## Solution

The question follows the Island pattern and is quite similar to Number of Islands problem.

We will traverse the matrix linearly to find islands.

Whenever we find a cell with the value '1' (i.e., land), we have found an island. Using that cell as the root node, we will perform a **Depth First Search** (**DFS**) or **Breadth First Search** (**BFS**) to find all of its connected land cells. During our DFS or BFS traversal, we will find and mark all the horizontally and vertically connected land cells.

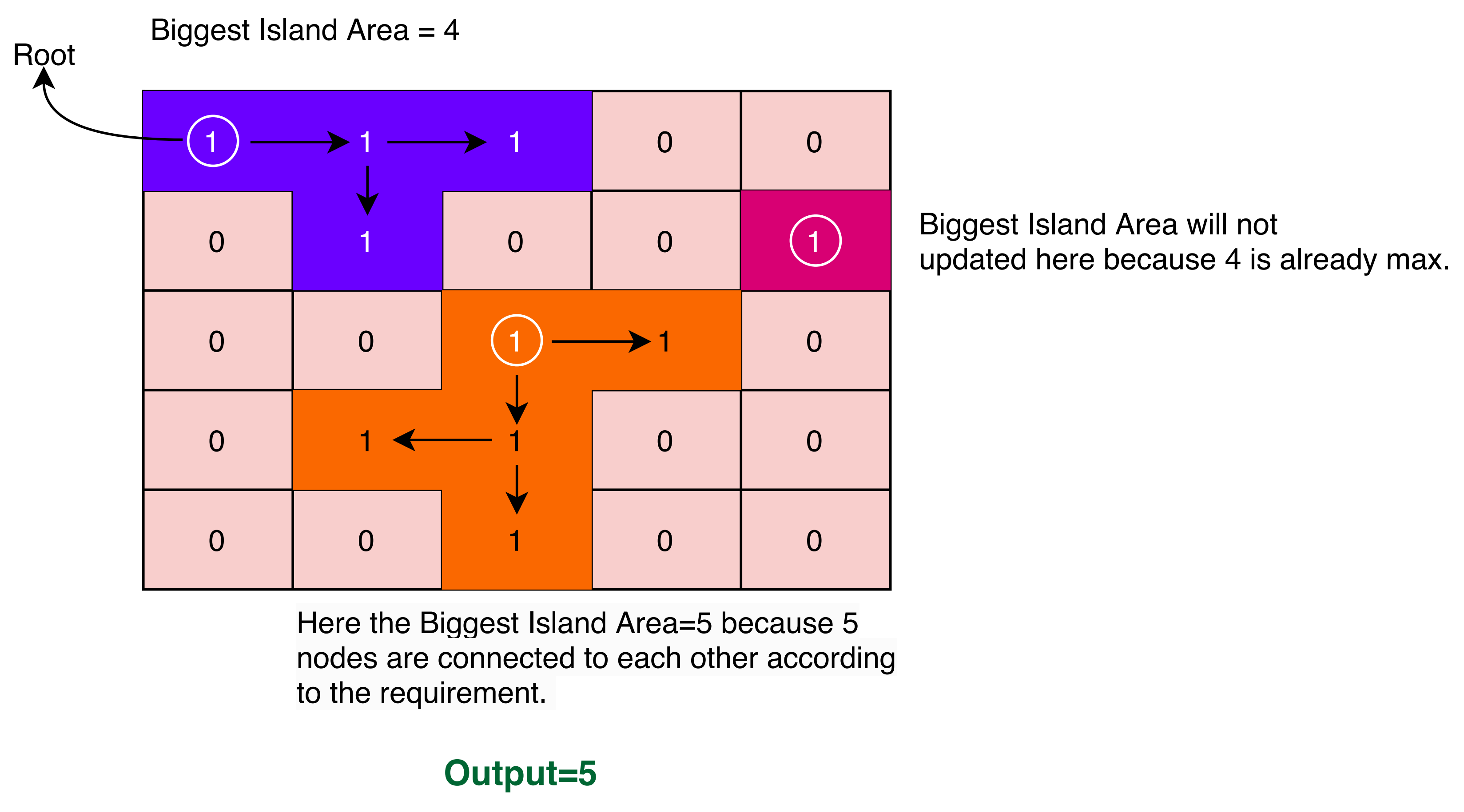
We will keep a variable to remember the max area of any island.

**Algorithm Walkthrough**

Here is the detailed walkthrough of the DFS algorithm:

1. We first initialize `biggestIslandArea` to 0. This variable will keep track of the largest island's area we have encountered so far.
2. Then, we traverse each cell in the matrix. If the cell's value is 1 (land), we begin a DFS search from this cell using the `visitIslandDFS` function. This function will visit the cell and all its neighboring cells that are part of the same island.
3. In the `visitIslandDFS` function, we first check if the current cell (x, y) is within the boundaries of the matrix and if it's a land cell. If it's not, we return 0.
4. We then mark the current cell as visited by setting its value to 0 (water). This helps avoid visiting the same cell again and ending up in an infinite loop.
5. We initialize the `area` of the current island to 1 (counting the current cell), and then add to it the areas returned by the recursive DFS calls for the neighboring cells (top, bottom, left, and right).
6. After we finish the DFS for a cell (meaning we have visited all cells in the island that the cell is a part of), we update `biggestIslandArea` with the maximum of its current value and the area of the island we just finished exploring.
7. Finally, after traversing all cells in the matrix, we return `biggestIslandArea`, which now holds the area of the largest island.

**Here is the visual representation of the algorithm**:



Biggest Island Area will not updated here because 4 is already max.

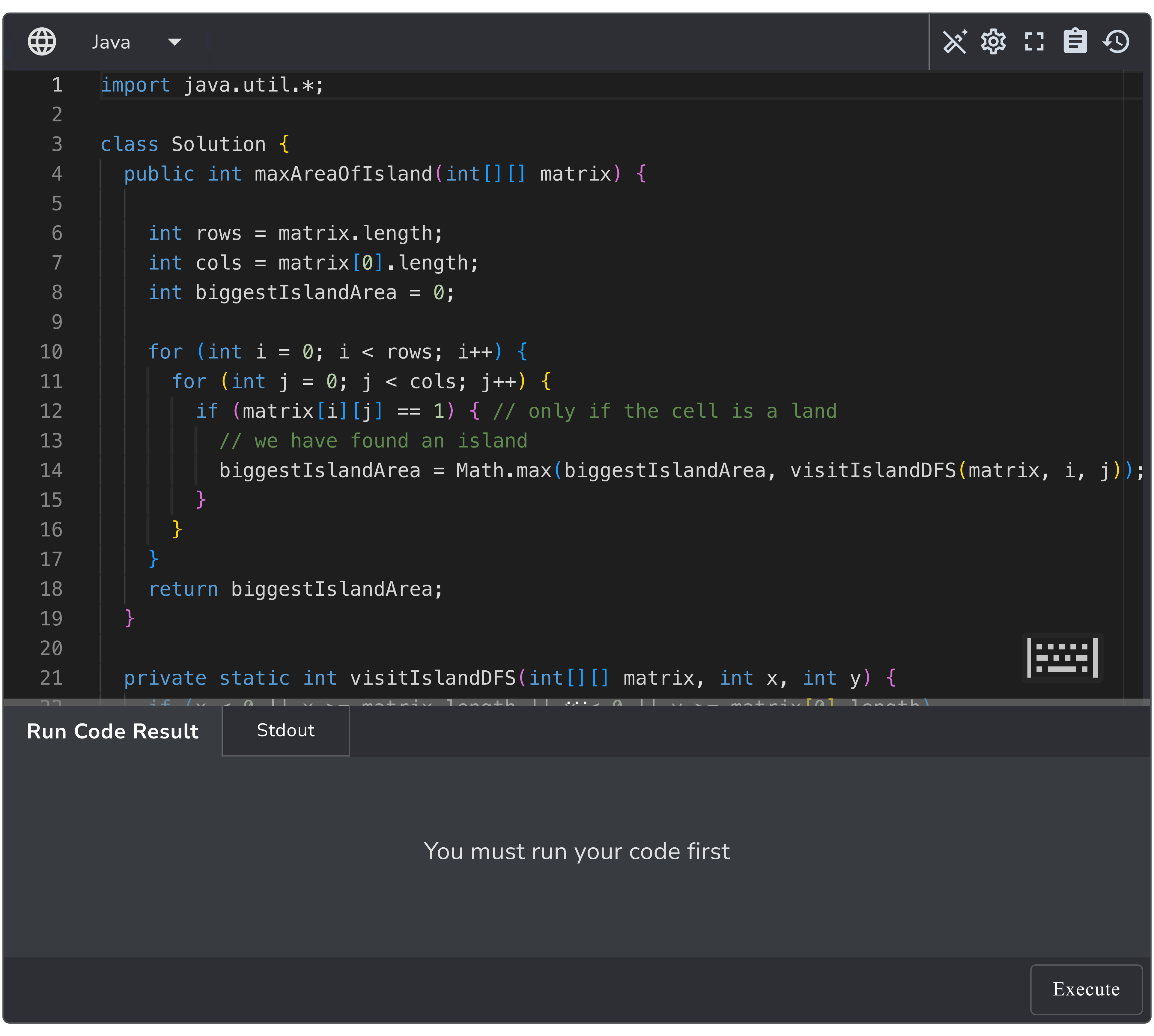
Here the Biggest Island Area=5 because 5 nodes are connected to each other according to the requirement.

**Output=5**

Biggest Island

## Code (DFS)

Here is what our DFS algorithm will look like. We will update the input matrix to mark nodes visited.



```java
import java.util.*;

class Solution {
  public int maxAreaOfIsland(int[][] matrix) {

    int rows = matrix.length;
    int cols = matrix[0].length;
    int biggestIslandArea = 0;

    for (int i = 0; i < rows; i++) {
      for (int j = 0; j < cols; j++) {
        if (matrix[i][j] == 1) { // only if the cell is a land
          // we have found an island
          biggestIslandArea = Math.max(biggestIslandArea, visitIslandDFS(matrix, i, j));
        }
      }
    }
    return biggestIslandArea;
  }

  private static int visitIslandDFS(int[][] matrix, int x, int y) {
```

You must run your code first

**Time Complexity**

Time complexity of the above algorithm will be $O(M * N)$, where 'M' is the number of rows and 'N' is the number of columns of the input matrix. This is due to the fact that we have to traverse the whole matrix to find islands.

**Space Complexity**

DFS recursion stack can go $M * N$ deep when the whole matrix is filled with '1's. Hence, the space complexity will be $O(M * N)$, where 'M' is the number of rows and 'N' is the number of columns of the input matrix.

← Previous
Biggest Island (Easy)
Next →
Flood Fill (Easy)
☐ Mark as Completed